# COMP438/538 Reinforcement Learning

## Bellman Equations and Dynamic Programming

Barış Akgün

Fall 2026

# So Far

- Learning from Experience
- The Interaction Loop: state-action-reward trajectories
- Policies and (discounted) returns
- State and State-Action Values as expected returns

- Formalizing it all: MDPs

- Where we left off?
  - How to pick between two policies?
  - How to calculate expected returns?
  - What to do when the trajectories do not end?

- Reminder: Episodic vs Continuing Problems (and terminal/absorbing states)

# Brainstorming

- Given a model of the environment and the policy, how can we calculate returns?

# Evaluating a policy

**Prediction** (policy evaluation): calculate a (fixed) policy's state and state-action values.

- Averaging returns
  - Sample Variance
  - Tail for continuing trajectories
- Using a model
  - Handles continuing trajectories
  - Leverages the Markovian nature of the problem

**Control**: find a policy with higher expected return.

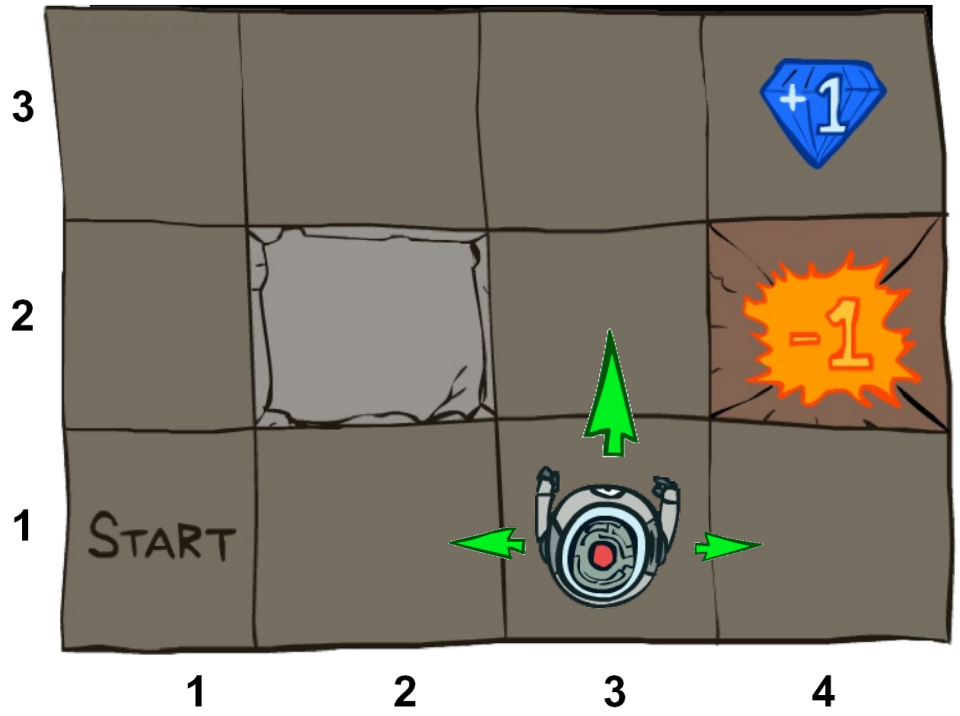
Values account for delayed consequences and help us compare decisions.

# Gridworld as an MDP

**Formulate this as an MDP:**

**What are the states, actions, model, rewards and discount?**

- States: Cells on the grid
- Actions: up, down, left, right
- Model:
  - Intended direction: 0.8
  - each perpendicular direction: 0.1
  - A blocked movement leaves the agent in the same cell
- Entering either terminal cell ends the episode; no rewards follow
- $\gamma = 0.5$ for today's examples




(4,3): **+1 on entry** (4,2): **−1 on entry**

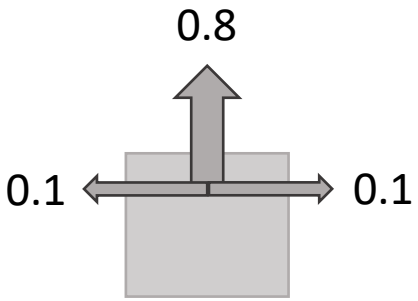
Other transitions: living reward −0.04

# A smaller grid for calculations

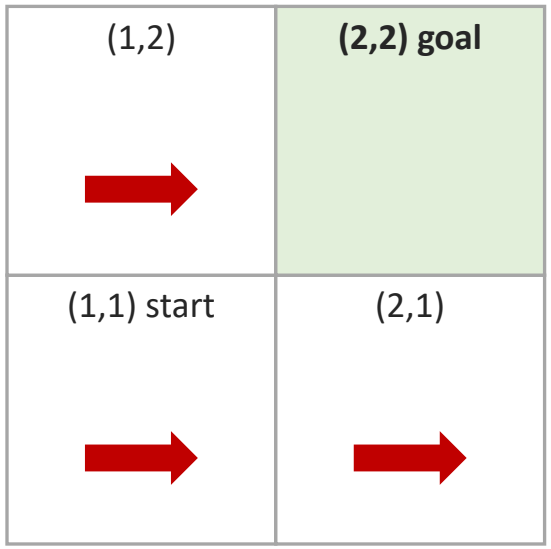
**Same movement rules, smaller map. This is a separate MDP.**

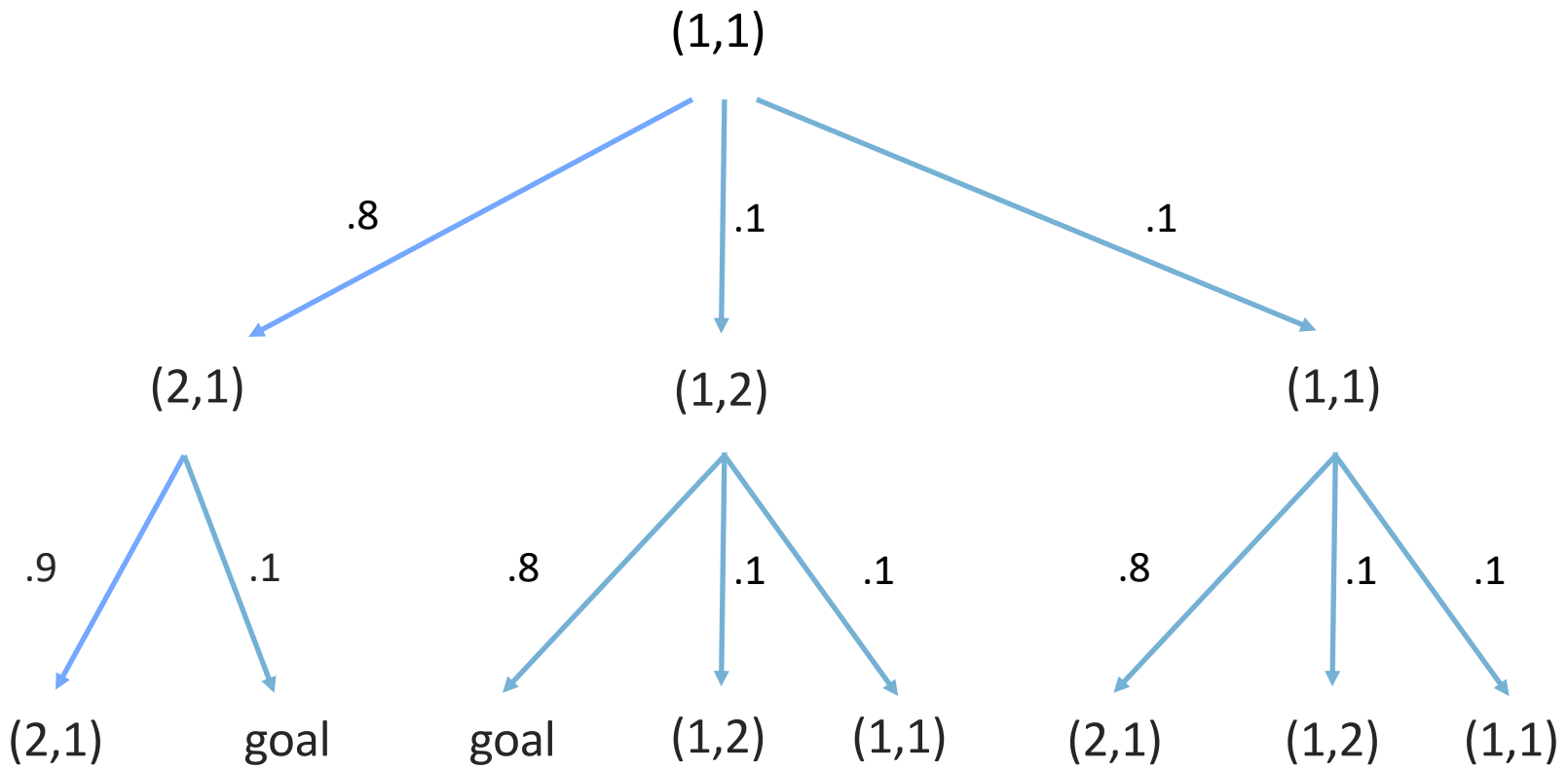
- Rewards:
  - Goal entry: **+1** total
  - Every other transition before termination: **–0.04**
- Goal continuation value 0; discount $\gamma = 0.5$
- Fixed policy $\pi$: choose **right** at every nonterminal cell

| | |
|---|---|
| (1,2) | **(2,2) goal** |
| (1,1) start | (2,1) |



# Two-step transition prediction




**Under the always-right policy, what is the probability of reaching the goal in two steps?**

# Reusing future subproblems

- Different routes can arrive at the same state.
- From that state, the same policy faces the same future problem.
- Store one value per state and reuse it when evaluating predecessors.

**Dynamic programming uses this repeated structure to compute values.**

# Back to Values

- State-Action Value definition: $q^{\pi}(s,a) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$
- Given a trajectory $\ldots S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, \ldots$, discounted return: $G_t = R_{t+1} + \gamma G_{t+1}$

$$\Rightarrow q^{\pi}(s,a) \doteq \mathbb{E}_{\pi}[\, R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a \,]$$
$$= \mathbb{E}[R_{t+1} | S_t = s, A_t = a] + \gamma \mathbb{E}_{\pi}[\, G_{t+1} \mid S_t = s, A_t = a \,]$$

- Recall that $\mathcal{R}(s,a) \doteq \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$
- With the Markov State Assumption:

$$\mathbb{E}_{\pi}[\, G_{t+1} \mid S_t = s, A_t = a \,] = \sum_{s' \in \mathcal{S}^+} \underbrace{\Pr\{S_{t+1} = s' | S_t = s, A_t = a\}}_{\mathcal{T}(s,a,s')} v^{\pi}(s')$$

# State and State-Action Values

- Plug everything back:

$$q^{\pi}(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^{+}} \mathcal{T}(s,a,s') v^{\pi}(s')$$

- Deterministic Policy ($a = \pi(s)$):

$$v^{\pi}(s) = q^{\pi}\big(s, \pi(s)\big)$$

- Stochastic Policy ($\pi(a \mid s) = \Pr\{A_t = a \mid S_t = s\}$):

$$v^{\pi}(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) q^{\pi}(s,a)$$

**Check:** if the agent acts differently after the first action, can that action's value change?

# Bellman Expectation Equation

$$v^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$
$$= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$
$$= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \sum_{s' \in \mathcal{S}^+, r} \mathcal{M}(s, a, r, s')[r + \gamma v^{\pi}(s')]$$

Equivalent form using expected reward and next-state probabilities:

$$v^{\pi}(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)[\mathcal{R}(s, a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s, a, s') v^{\pi}(s')]$$

**Expected immediate reward + discounted expected continuation**

Deterministic Policy :

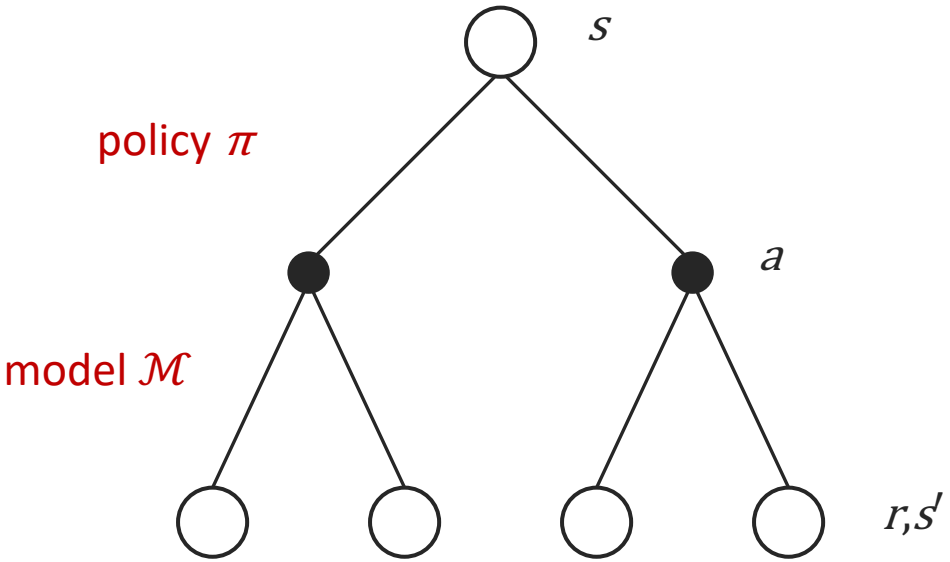
$$v^{\pi}(s) = \mathcal{R}(s, \pi(s)) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s, \pi(s), s') v^{\pi}(s')$$

# Action-value recursion and backups

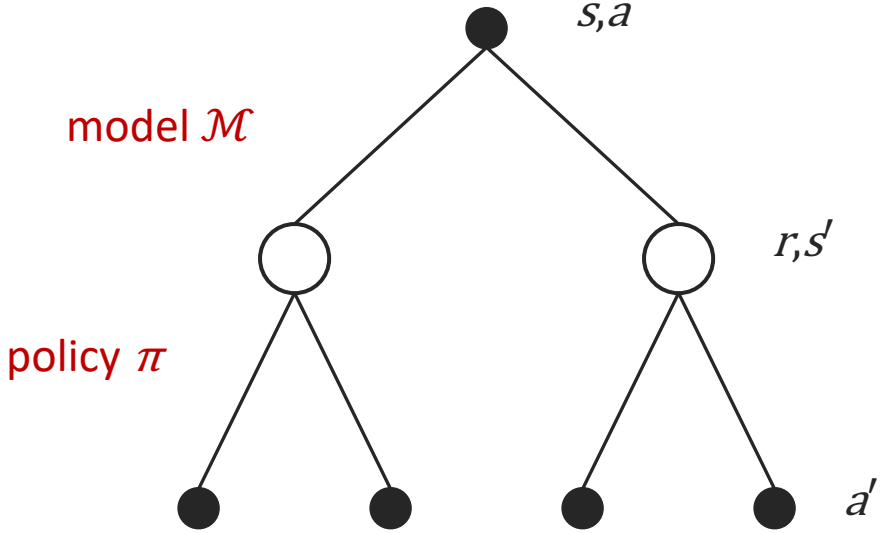
$$q^{\pi}(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{T}(s,a,s') \sum_{a' \in \mathcal{A}(s')} \pi(a' \mid s') q^{\pi}(s',a')$$

**A backup carries value information from possible successors to the state or action being evaluated.**

**State value $v^{\pi}(s)$**

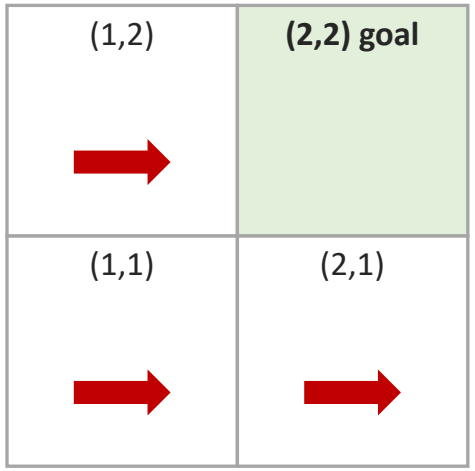



**Action value $q^{\pi}(s,a)$**



# Three states, three value equations

- Policy $\pi$: always right. Goal continuation value 0.
- The paths may be arbitrarily long; this policy has only **three** unknown nonterminal values.




$$v^{\pi}((1,1)) = -0.04 + 0.5[0.1v^{\pi}((1,1)) + 0.1v^{\pi}((1,2)) + 0.8v^{\pi}((2,1))]$$

$$v^{\pi}((1,2)) = ?$$

$$v^{\pi}((2,1)) = ?$$

# Solving the value equations

**Fixed policy: a linear system for the unknown state values.**

$$v^{\pi}((1,1)) = -0.04 + 0.5[0.1v^{\pi}((1,1)) + 0.1v^{\pi}((1,2)) + 0.8v^{\pi}((2,1))]$$

$$v^{\pi}((1,2)) = 0.792 + 0.5[0.1v^{\pi}((1,1)) + 0.1v^{\pi}((1,2))]$$

$$v^{\pi}((2,1)) = 0.064 + 0.5[0.9v^{\pi}((2,1))]$$

Optional Home Exercise: Write the system in matrix form and solve it.

Question: What is the complexity of this way of performing policy evaluation for an MDP with $n$ states?

# Iterative policy evaluation

$$V_{k+1}(s) \leftarrow \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)[\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') V_k(s')]$$

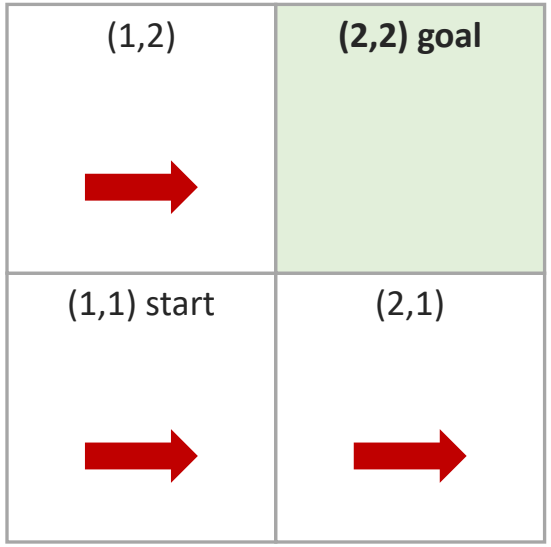
- Initialize $V_0(s) = 0$, including the terminal state.
- Compute every new entry using the **old values**.
- After all states are updated, replace the table and repeat.
- Stop when the largest change over a sweep is *sufficiently* small.

$k$ **counts computational sweeps. The policy stays fixed.**

Complexity per step?

Total complexity?

# First evaluation sweep

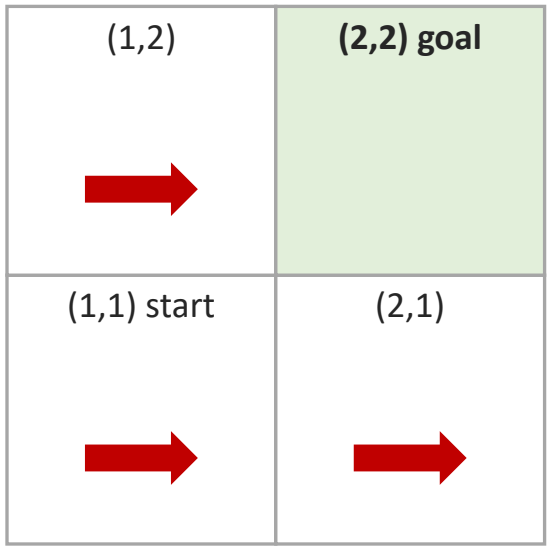



| Sweep | (1,1) | (1,2) | (2,1) | Terminal |
|---|---|---|---|---|
| $V_0$ | 0 | 0 | 0 | 0 |
| $V_1$ | ? | ? | ? | 0 |

**With zero initial continuation values, the first sweep gives expected immediate rewards.**

- Always-right policy, $\gamma = 0.5$, $V_0 = 0$.

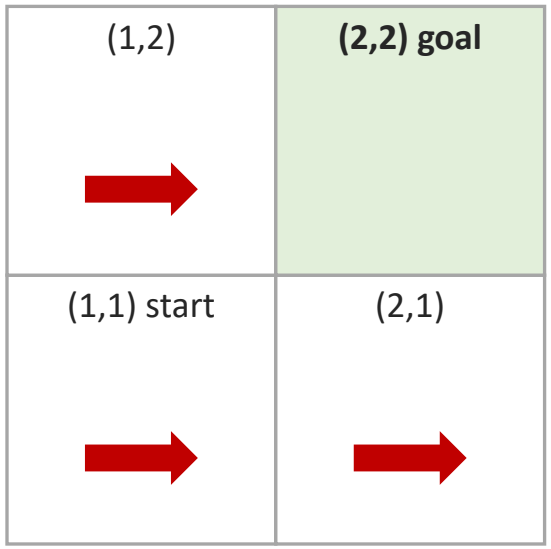
# Second evaluation sweep




| Sweep | (1,1) | (1,2) | (2,1) | Terminal |
|---|---|---|---|---|
| $V_1$ | −0.04 | 0.792 | 0.064 | 0 |
| $V_2$ | ? | ? | ? | 0 |

**Compute $V_2$ using $V_1$ for every successor value. Keep the policy fixed.**

# What the sweeps mean




| Sweep | (1,1) | (1,2) | (2,1) | Terminal |
|---|---|---|---|---|
| $V_1$ | −0.04 | 0.792 | 0.064 | 0 |
| $V_2$ | 0.0232 | 0.8296 | 0.0928 | 0 |
| $V_3$ | 0.03976 | 0.83464 | 0.10576 | 0 |

**Starting from $V_0 = 0$, $k$ synchronous sweeps give the exact expected discounted sum of the first $k$ rewards under this policy.**

# Why repeated backups settle

**Bellman (expectation) operator** $B^\pi$: maps any value table $V$ to a new table

$$(B^\pi V)(s) \doteq \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)[\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s')V(s')]$$

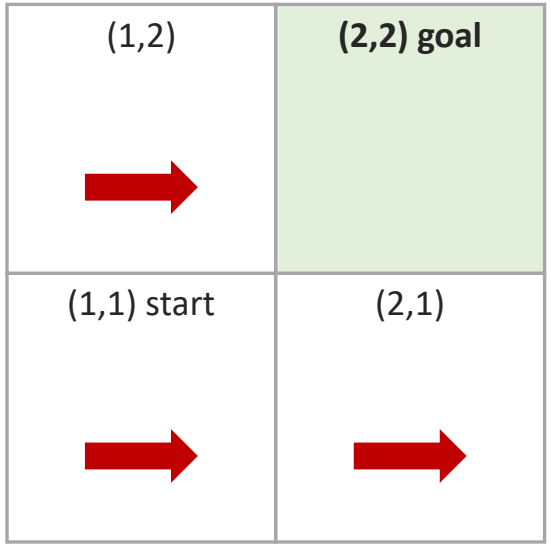
$$V_{k+1} = B^\pi V_k, \qquad B^\pi v^\pi = v^\pi$$

- $B^\pi$ applies one full policy-evaluation sweep.
- Probability averaging cannot increase the largest discrepancy; discounting multiplies it by $\gamma$.

$$\max_{s \in \mathcal{S}^+} |V_{k+1}(s) - v^\pi(s)| \leq \gamma \max_{s \in \mathcal{S}^+} |V_k(s) - v^\pi(s)|$$

**Finite MDP, bounded rewards, fixed policy, $0 \leq \gamma < 1$: the sweeps converge to its unique value function.**

# A better first action?




**At (2,1), compare continuing right with choosing up once, then following the same policy.**

$$v^{\pi}((2,1)) = q^{\pi}((2,1), \text{right}) = 32/275 \approx 0.11636$$

$$q^{\pi}((2,1), \text{up}) = 0.792 + 0.5[0.1(14/275) + 0.1(32/275)] \approx 0.80036$$

Question: How to formalize this step?

# Optimal values

$$v^*(s) = \max_\pi v^\pi(s), \qquad q^*(s,a) = \max_\pi q^\pi(s,a)$$

- $v^*(s)$: best expected return from $s$.
- $q^*(s,a)$: expected return after first taking $a$, then acting optimally.

$$v^*(s) = \max_{a \in \mathcal{A}(s)} q^*(s,a)$$

# Bellman optimality equations

$$v^*(s) = \max_{a \in \mathcal{A}(s)} [\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') v^*(s')]$$

$$q^*(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{T}(s,a,s') \max_{a' \in \mathcal{A}(s')} q^*(s',a')$$

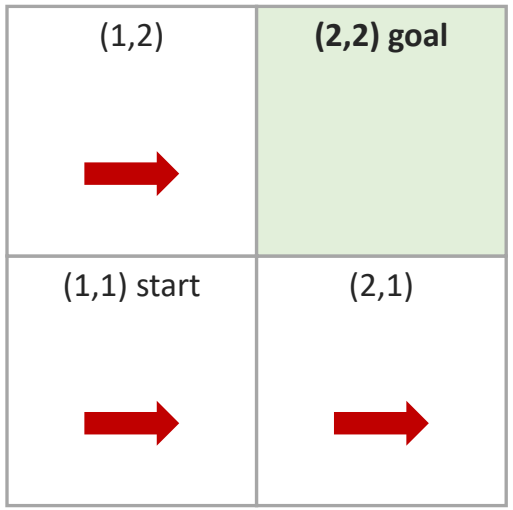
$$\pi^*(s) = \arg\max_{a \in \mathcal{A}(s)} q^*(s,a)$$

**The agent chooses an action and the environment's outcomes are still averaged.**

Side Note: For finite discounted MDPs, there always exists an optimal deterministic policy.

**Next**: policy improvement, value iteration and policy iteration.

# Improving the policy




At (2,1), choosing up once and then following always-right gives higher expected return:

$$q^{\pi}((2,1),\text{up}) \approx 0.80036 \quad > \quad v^{\pi}((2,1)) \approx 0.11636$$

**Does choosing better actions at every visit give a better policy?**

Today: policy improvement, policy iteration and value iteration.

# Greedy policy improvement

Use the current policy's values to compare candidate first actions (no need to store $q^\pi(s,a)$):

$$q^\pi(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') v^\pi(s')$$

Choose a maximizing action at each nonterminal state:

$$\pi'(s) = \arg\max_{a \in \mathcal{A}(s)} q^\pi(s,a)$$

For a deterministic current policy,

$$q^\pi(s,\pi'(s)) = \max_a q^\pi(s,a) \geq q^\pi(s,\pi(s)) = v^\pi(s)$$

**"Greedy" includes expected future return through $v^\pi$.**

# Why policy improvement works

If at every nonterminal state,

$$q^{\pi}(s, \pi'(s)) \geq v^{\pi}(s)$$

then following $\pi'$ throughout is at least as good:

$$v^{\pi'}(s) \geq v^{\pi}(s)$$

**Why?** Replace the old continuation with another improved action:

$$v^{\pi}(s) \leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma v^{\pi}(S_{t+1}) \mid S_t = s]$$
$$\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v^{\pi}(S_{t+2}) \mid S_t = s]$$
$$\leq \cdots \rightarrow v^{\pi'}(s)$$

Finite MDP, bounded rewards, $0 \leq \gamma < 1$.

# Policy iteration

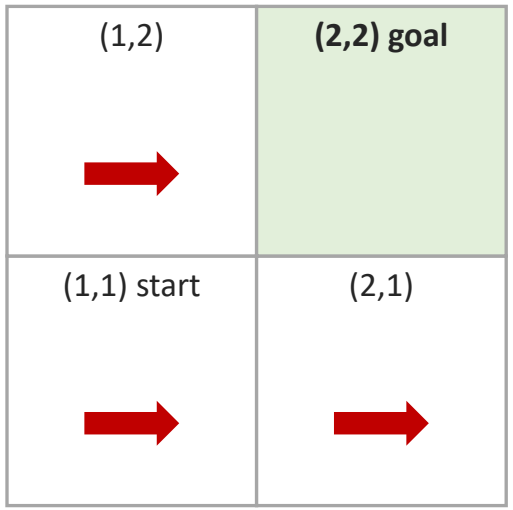
1. Choose an initial policy.
2. **Evaluate:** calculate its values while keeping the policy fixed.
3. **Improve:** choose greedy actions while keeping those values fixed.
    - Retain the current action when it is among the maximizing actions.
4. Repeat evaluation and improvement until the policy is stable.

At stability, with exact evaluation,

$$v^{\pi}(s) = \max_{a \in \mathcal{A}(s)} [\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') v^{\pi}(s')]$$

This is the Bellman optimality equation: $v^{\pi} = v^{*}$.

# Improving always-right on the grid




**Solved values for $\pi$ = always-right** (+1 on goal entry, −0.04 otherwise, $\gamma = 0.5$)

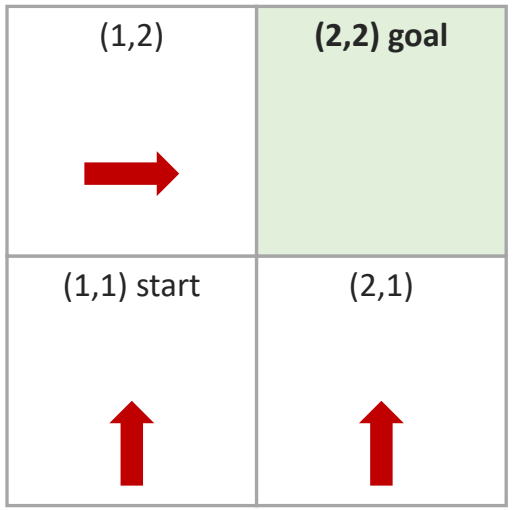
| State | $v^\pi(s)$ |
|---|---|
| **(1,1)** | $14/275 \approx 0.05091$ |
| **(1,2)** | $230/275 \approx 0.83636$ |
| **(2,1)** | $32/275 \approx 0.11636$ |

**Calculate $q^\pi((1,1), \text{up})$. Is up better than right here?**

$$q^\pi((1,1), \text{up}) = -0.04 + 0.5[0.8(46/55) + 0.1(32/275) + 0.1(14/275)] \approx 0.30291 > v^\pi((1,1))$$

Do this for all three nonterminal states at home.

# The improved policy and its values




Action scores under the **old** continuation policy:

| State | up | right | down | left |
|---|---|---|---|---|
| **(1,1)** | **0.30291** | 0.05091 | −0.01127 | 0.02473 |
| **(1,2)** | 0.44036 | **0.83636** | 0.12618 | 0.33891 |
| **(2,1)** | **0.80036** | 0.11636 | 0.01491 | 0.09018 |

Reevaluate the changed policy $\pi'$ = (up, right, up):

$$v^{\pi'}((1,1)) = 199/550 \approx 0.36182, \qquad v^{\pi'}((1,2)) = v^{\pi'}((2,1)) = 469/550 \approx 0.85273$$

**A further improvement leaves the policy unchanged; up and right tie at (1,1).**

# Why shorten policy evaluation?

- A full policy evaluation may require many sweeps.
- Useful action choices may emerge before the values have settled.

**Must we finish evaluating each policy before making another improvement?**

Value iteration combines a greedy choice with one backup using the current table.

# Value iteration

For every nonterminal state $s \in \mathcal{S}$,

$$V_{k+1}(s) \leftarrow \max_{a \in \mathcal{A}(s)} [\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') V_k(s')]$$

1. Initialize $V_0$ to zero, including terminal entries.
2. Compute each new nonterminal entry using the **old** table. (implementation requires 2 copies!)
3. Replace the table after the full sweep; keep terminal entries zero.
4. Repeat until the largest sweep change is sufficiently small.

Extract a greedy policy using the final stored table $V$:

$$\pi(s) = \arg\max_{a \in \mathcal{A}(s)} [\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') V(s')]$$

# A value-iteration sweep



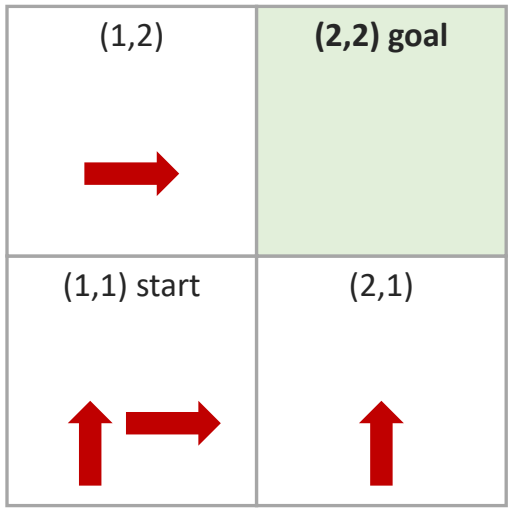

$\gamma = 0.5$; living reward −0.04; +1 on goal entry, continuation 0; synchronous VI from $V_0 = 0$.

| Sweep | (1,1) | (1,2) | (2,1) | Terminal |
|---|---|---|---|---|
| $V_1$ | −0.04 | 0.792 | 0.792 | 0 |
| $V_2$ | ? | ? | ? | 0 |

**Compute $V_2((1,1))$ and identify the maximizing actions.**

- Every candidate action uses the same $V_1$ successor values.

# Value-iteration results




At (1,1): up/right each score **0.3144**; down/left each score **–0.0184**.

$$V_2((1,1)) = -0.04 + 0.5[0.8(0.792) + 0.1(0.792) + 0.1(-0.04)] = 0.3144$$

| Table | (1,1) | (1,2) | (2,1) | Terminal |
|---|---|---|---|---|
| $V_1$ | −0.04 | 0.792 | 0.792 | 0 |
| $V_2$ | 0.3144 | 0.8296 | 0.8296 | 0 |
| $V_3$ | 0.34904 | 0.8492 | 0.8492 | 0 |
| $v^*$ | 0.36182… | 0.85273… | 0.85273… | 0 |

**Values keep changing even when the greedy actions agree -> Policy Converges before the values**

If you do not care about the values, you can check for policy convergence in Value Iteration to have the best of both worlds. But you need to do policy evaluation, policy improvement and do a convergence check.

# Q-value iteration

Use action-value estimates directly:

$$Q_{k+1}(s,a) \leftarrow \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{T}(s,a,s') \max_{a' \in \mathcal{A}(s')} Q_k(s',a')$$

- $Q_k(s,a)$: action-value estimate after $k$ sweeps.
- Greedy action selection from $Q$ needs no model.
- Computing this expected update still needs the model.

**In a couple of weeks: Q-learning uses sampled transitions for this kind of target.**

# Policy iteration and value iteration

| | Policy iteration | Value iteration |
|---|---|---|
| **Main computation** | Evaluate a fixed policy, then improve | Max backup at each state, each sweep |
| **Intermediate table** | Evaluates the current policy, exactly or approximately | May not be a policy's infinite-horizon values |
| **Work per outer iteration** | Evaluation work plus greedy improvement | One sweep over states and candidate actions |
| **On our grid** | One improving round, then reevaluation and stability | $V_1,\ V_2,\ V_3$ approach the same $v^*$ |

**A PI round and a VI sweep measure different amounts of work.**

# Which one to choose?

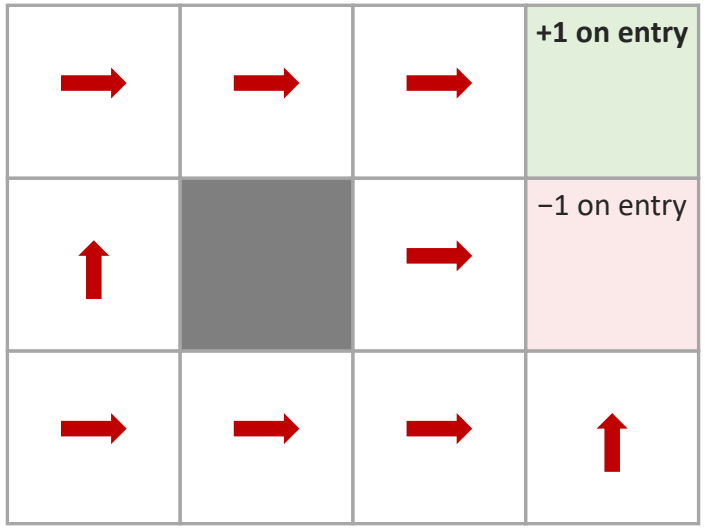
| Situation | Tends to favour | Why |
|---|---|---|
| **Many actions per state** | Policy iteration | Evaluation sweeps use one action per state; all actions are compared only at improvement |
| **Many states** | Value iteration, or PI with a few evaluation sweeps | An exact solve costs about $\|\mathcal{S}\|^3$; a sweep costs about $\|\mathcal{S}\|^2\|\mathcal{A}\|$ (less with sparse transitions) |
| **Discount close to 1** | Policy iteration | VI's worst-case error bound shrinks only by a factor $\gamma$ per sweep; PI usually needs few improvement rounds |
| **Need a certified optimal policy** | Policy iteration | Stops at a stable policy (its stability test) |

**No universal winner: count the total work.** PI with a few evaluation sweeps per improvement sits in between (generalized policy iteration).

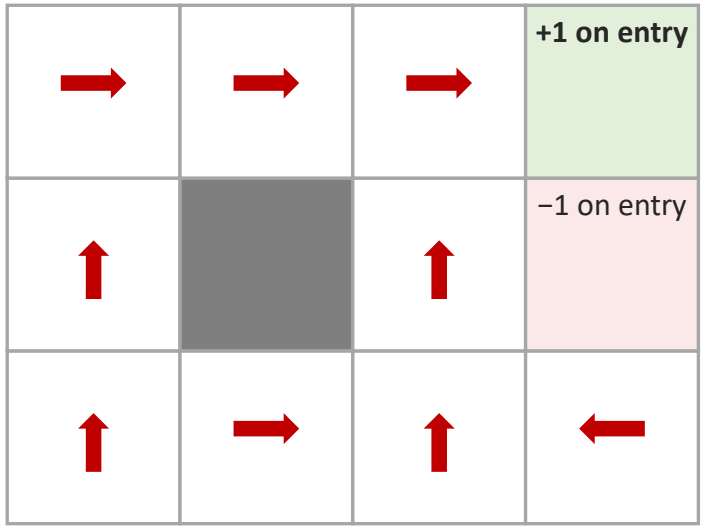
# What is optimal depends on the reward

Same 3×4 grid and motion model. **For this comparison,** $\gamma = 0.9$**:** later rewards count more, which makes the risk/delay trade-off more visible than with $\gamma = 0.5$.

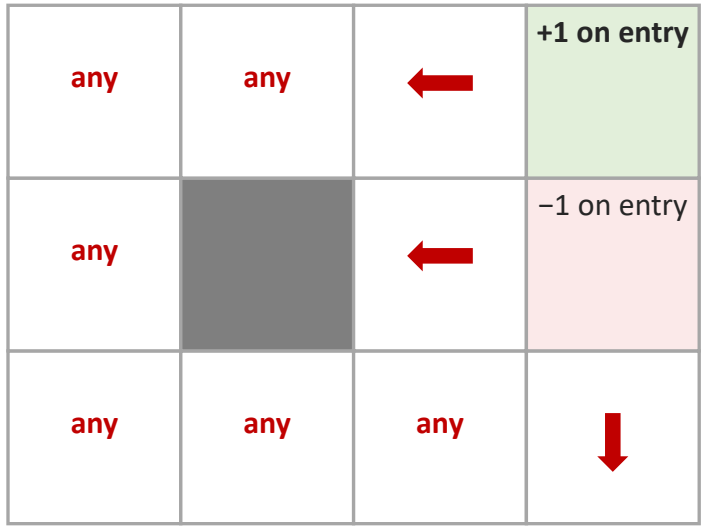
**Living reward −2**



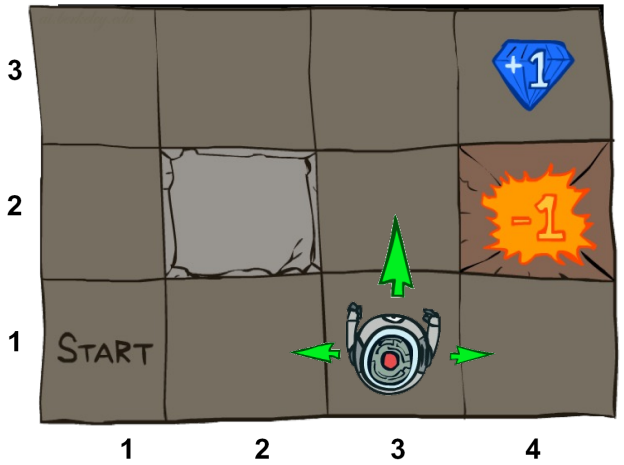

**Living reward −0.04**




**Living reward +0.2**




- At (3,2), should the policy choose right toward the −1 terminal, up, or left?
- Can a reward make avoiding the +1 terminal optimal?



# Generalized policy iteration




Generalized policy iteration describes how these processes interact, at different granularities.

- PI completes evaluation before improvement; VI combines improvement with a single backup.
- In a finite discounted MDP, exact-model asynchronous updates can also converge when every state continues to be updated.

**In a few weeks, we will do evaluation and improvement using experience (environment interaction).**

# Planning with a known model

| Task | Computation |
| --- | --- |
| **Evaluate a supplied policy** | Bellman expectation backups |
| **Find a better policy** | Greedy improvement using that policy's values |
| **Solve control with a known model** | Policy iteration or value iteration |

Which computations need the transition and reward model?

Which action-selection step can use a learned table directly?

**Next**: estimating values from sampled experience when the model is unavailable.

# Appendix: other forms of the VI updates

**Deterministic policy** $a = \pi(s)$:

$$V_{k+1}(s) \leftarrow \max_{a \in \mathcal{A}(s)} [\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') V_k(s')]$$

$$\pi'(s) = \arg\max_{a \in \mathcal{A}(s)} [\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') v^\pi(s')]$$

**Reward depending on the next state:** $\mathcal{R}(s,a,s') \doteq \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s']$, so $\mathcal{R}(s,a) = \sum_{s'} \mathcal{T}(s,a,s') \mathcal{R}(s,a,s')$

$$V_{k+1}(s) \leftarrow \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') [\mathcal{R}(s,a,s') + \gamma V_k(s')]$$

$$\pi'(s) = \arg\max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') [\mathcal{R}(s,a,s') + \gamma v^\pi(s')]$$

Our grids fit this form: the reward depends only on the next cell (+1 or −1 on entry, −0.04 otherwise).

For the PI updates, remove the $\max$ operator from the value calculations and use $\pi(s)$ instead of $a$